Numerical Integration in C:

Numerical integration approximates the integral of a function over an interval when the exact solution is difficult or impossible to find. In this tutorial, we will implement numerical integration using the **Trapezoid Rule** and **Simpson's Rule** based on the Newton-Cotes formulas. This guide is written for beginners in programming and numerical methods.

Prerequisites

1. **Basic Programming Knowledge:** Familiarity with C programming basics (variables, loops, functions).
2. **Mathematics:** Basic understanding of integration and functions.

Understanding the Concepts

The Problem

We want to approximate the integral of a function $f(x)$ over an interval $[a, b]$:

$$\int_a^b f(x)\,dx$$

Newton-Cotes Methods

Newton-Cotes methods approximate $f(x)$ using polynomials. The most common ones are:

1. **Trapezoid Rule:** Approximates $f(x)$ with a straight line.
2. **Simpson's Rule:** Approximates $f(x)$ with a parabola.

Trapezoid Rule

The formula for the Trapezoid Rule is:

$$\int_a^b f(x)\,dx \approx \frac{h}{2}[f(a) + f(b)]$$

where $h = b - a$.

For better accuracy, we divide $[a, b]$ into n smaller intervals (subdivisions) and apply the rule to each subinterval. This is called the **Composite Trapezoid Rule**:

$$\int_a^b f(x)\,dx \approx \frac{h}{2}\left[f(a) + 2\sum_{i=1}^{n-1} f(x_i) + f(b)\right]$$

where $h = \frac{b-a}{n}$ and $x_i = a + i \cdot h$.

Step 2: Implement the Trapezoid Rule

The Trapezoid Rule calculates the integral for one interval. Extend this to multiple intervals for more accuracy.

Create a function $f(x)$ that you want to integrate. For example, let's find the area under the curve of between 0 and 1.

```
# include <stdio.h>
# include <math.h>

// Define the function to integrate
double f(double x) {
    return x * x; // Example: f(x) = x^2
}

// Trapezoid Rule for one interval
double trapezoid(double a, double b) {
    return (b - a) * (f(a) + f(b)) / 2;
}

// Composite Trapezoid Rule
double trapezoid_rule(double a, double b, int n) {
    double h = (b - a) / n;
    double sum = f(a) + f(b);

    for (int i = 1; i < n; i++) {
        double x = a + i * h;
        sum += 2 * f(x);
    }

    return (h / 2) * sum;
}

int main() {
    double a = 0, b = 2; // Time interval
    int n = 4; // Number of intervals

    double distance_trap = trapezoid_rule(a, b, n);
    printf("Distance using Trapezoid Rule: %lf m\n", distance_trap);

    return 0;
}
```

Simpson's Rule

Simpson's Rule uses a quadratic polynomial. The formula is:

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right]$$

Simpson's Rule approximates an integral by fitting a quadratic polynomial (parabola) to three points: the left endpoint (a), the midpoint, and the right endpoint (b) of an interval. To extend this method to a composite rule over multiple subintervals, each segment must contain exactly three points: two endpoints and one midpoint.

Composite Simpson's Rule:

1. The interval $[a, b]$ is divided into n subintervals, each of width $h = \frac{b-a}{n}$.
2. To ensure there is a midpoint for every pair of subintervals, n must be even.
   - For example, with $n = 4$, the points are:
     $$x_0, x_1, x_2, x_3, x_4$$
     where x_0 and x_4 are endpoints, and x_2 is a midpoint for the interval formed by x_1 and x_3.

If n is odd, the segmentation cannot maintain the required midpoint structure across all intervals, breaking the method's assumptions and invalidating the results.

For more accuracy, we use the **Composite Simpson's Rule**, which requires n to be even:

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\left[f(a) + 4\sum_{i=1,\text{odd}}^{n-1} f(x_i) + 2\sum_{i=2,\text{even}}^{n-2} f(x_i) + f(b)\right]$$

where $h = \frac{b-a}{n}$.

Step 3: Implement Simpson's Rule

Simpson's Rule requires n to be even. Implement the basic formula and its composite version.

```
// Simpson's Rule for one interval
double simpson(double a, double b) {
    double mid = (a + b) / 2;
    return (b - a) * (f(a) + 4 * f(mid) + f(b)) / 6;
}

// Composite Simpson's Rule
double simpsons_rule(double a, double b, int n) {
    if (n % 2 != 0) {
        printf("n must be even for Simpson's Rule.\n");
        return -1;
    }

    double h = (b - a) / n;
    double sum = f(a) + f(b);
```

```
    for (int i = 1; i < n; i++) {
        double x = a + i * h;
        if (i % 2 == 0) {
            sum += 2 * f(x);
        } else {
            sum += 4 * f(x);
        }
    }

    return (h / 3) * sum;
}
```

Real-Life Examples

Example 1: Estimating Distance Traveled

Suppose a car's velocity $v(t) = 3 \times t^{1.5}$ (in m/s) changes over time t (in seconds). To estimate the distance traveled from $t = 0$ to $t = 2$:

```
int main() {
    double a = 0, b = 2; // Time interval
    int n = 4; // Number of intervals

    double distance_trap = trapezoid_rule(a, b, n);
    printf("Distance using Trapezoid Rule: %lf m\n", distance_trap);

    double distance_simp = simpsons_rule(a, b, n);
    if (distance_simp != -1) {
        printf("Distance using Simpson's Rule: %lf m\n", distance_simp);
    }

    return 0;
}
```

Example 2: Estimating Work Done

If a variable force $F(x) = x^2 + 1$ acts over a distance, the work done is $\int_a^b F(x)\,dx$. Calculate the work done from $x = 0$ to $x = 3$:

```
int main() {
    double a = 0, b = 3; // Distance interval
    int n = 6; // Number of intervals
```

```
    double work_trap = trapezoid_rule(a, b, n);
    printf("Work using Trapezoid Rule: %lf J\n", work_trap);

    double work_simp = simpsons_rule(a, b, n);
    if (work_simp != -1) {
        printf("Work using Simpson's Rule: %lf J\n", work_simp);
    }

    return 0;
}
```

Key Points

1. **Function Definition:** Replace `f` with your desired function.
2. **Interval Selection:** Ensure n is even for Simpson's Rule.
3. **Experiment:** Try with different functions like exponential growth or oscillatory motion.
4. **Real-Life Applications:** Use these methods to solve practical problems in physics and engineering.

```
Start coding or generate with AI.
```